

Tema 1 – Sokoban

Vlad Schiopu – 332CB

1. Introducere

În această lucrare, am implementat și analizat doi algoritmi de căutare pentru rezolvarea jocului Sokoban: IDA* (Iterative Deepening A*) și Simulated Annealing. Scopul a fost de a compara eficiența lor în funcție de timpul de execuție, numărul de stări explorate și calitatea soluției, accentuând minimizarea mișcărilor de tip pull.

Am creat în clasa solver funcțiile solve, care rezolvă un nivel în funcție de algoritmul specificat și parametrii necesari fiecărui algoritm și visualize_solution care aplică fiecare mișcare din soluție hărții initiale, salvează imaginea la fiecare pas și creează un gif în final cu toate imaginile.

Am folosit fișierul Jupiter notebook main.ipynb pentru a arăta rezolvarile ambilor algoritmi pe o hartă la alegere (super_hard_map). Fiecare rezolvare este salvată în directorul images, dar și într-un folder specific pentru algoritm (images_ida_star sau images_simulated_annealing). Apoi am rulat ambii algoritmi pe toate hărțile și am salvat date despre numărul de stări explorate, timpul de execuție, lungimea soluției și numărul de mișcări de pull și am creat un dataframe pentru a realiza grafice. Am creat apoi încă un dataframe pentru a compara performanța în funcție de euristici.

2. Euristici folosite

Euristici simple:

Am început rezolvarea folosind ca euristica distanța Manhattan. Am calculat distanțele dintre cutii și target-uri folosind o abordare simplă, care poate asocia mai multe cutii cu aceeași tinta, și distanța dintre player și cea mai apropiată cutie care nu se află pe un target.

Euristici avansate:

Pentru a optimiza timpul de execuție pentru hărți mai complexe am schimbat algoritmul de calcul al distanțelor între cutii și target-uri cu algoritmul ungar (<https://brilliant.org/wiki/hungarian-matching/>) pentru a găsi cea mai bună configurație.

Am adăugat o euristica care analizează zonele de pe hartă folosind algoritmul flood fill (BFS). Verifică dacă în fiecare zonă există un număr egal de cutii și target-uri și adaugă o penalizare pentru fiecare cutie care nu poate ajunge la un target în zona ei (<https://sokoban.dk/wp-content/uploads/2016/02/Timo-Virkkala-Solving-Sokoban-Masters-Thesis.pdf>, pagina 28).

Pentru a obtine solutii mai calitative am adaugat o penalizare pentru miscarile de pull, astfel daca ultima miscare aplicata a fost una de pull maresc costul pentru a scadea prioritatea miscarii.

3. Algoritmi implementați si optimizari

Am inclus pentru ambii algoritmi optiunea use_pull_moves pentru a decide daca algoritmii au voie sa foloseasca miscari de pull sau nu.

-IDA* (Iterative Deepening A*)

Functia search:

- verific daca starea curenta este o solutie
- calculez valoarea functiei $f(stare) = g(stare) + h(stare)$
- daca functia depaseste limita returnez valoarea functiei (costul) pentru a actualiza limita in functia solve la urmatoarea iteratie
- generez toate mutarile posibile si le sortez dupa valoarea euristica
- aplic fiecare mutare hartii si apoi continui recursive cautarea pana gasesc solutia

Functia solve:

- setez limita initiala la valoarea euristica a starii initiale
- execut search cu limita curenta
- daca rezultatul este o lista, am gasit solutia, altfel am gasit o noua limita si continui cautarea cu noua limita

-Simulated Annealing

Algoritmul primeste ca parametrii:

- numarul maxim de iteratii (200000)
- o temperature initiala (3000)
- o rata de racire (0.999)
- o limita de timp (5 minute)
- un prag de restart (dupa 50 de mutari in care starea nu s-a imbunatatit se da restart cu sanse egale intre cea mai buna stare gasita si starea initiala)

Functia solve:

- incep cu starea initiala si ii calculez energia dupa functia euristica si setez temperature initiala

- execut intr-o bucla pana la numarul maxim de iteratii codul pentru gasirea solutiei

- generez o miscare folosind functia `get_weighted_move` care incearca fiecare miscare si ii atribuie un rating folosind functia euristica. Am adaugat un element random pentru a evita blocajul in maxime locale

- calculez energia noii stari si decid daca accept miscarea folosind functia `acceptance_probability` care accepta intotdeauna stari mai bune, si uneori stari mai proaste cu o probabilitate care scade odata cu temperatura

- actualizez cea mai buna stare si scad temperature

- fac restart in cazul in care solutia nu a fost imbunatatita intr-un anumit numar de miscari. Aici am mai incercat o varianta in care pragul este incrementat la fiecare restart pentru a fi sigur ca algoritmul gaseste eventual o solutie

-Optimizari realizate

Pentru a detecta din timp starile blocante am creat 2 functii care detecteaza 2 tipuri de deadlock:

- `find_corner_deadlocks` care verifica daca exista cutii care ating simultan 2 pereti si nu se afla pe un target (folosita mereu in abii algoritmi)

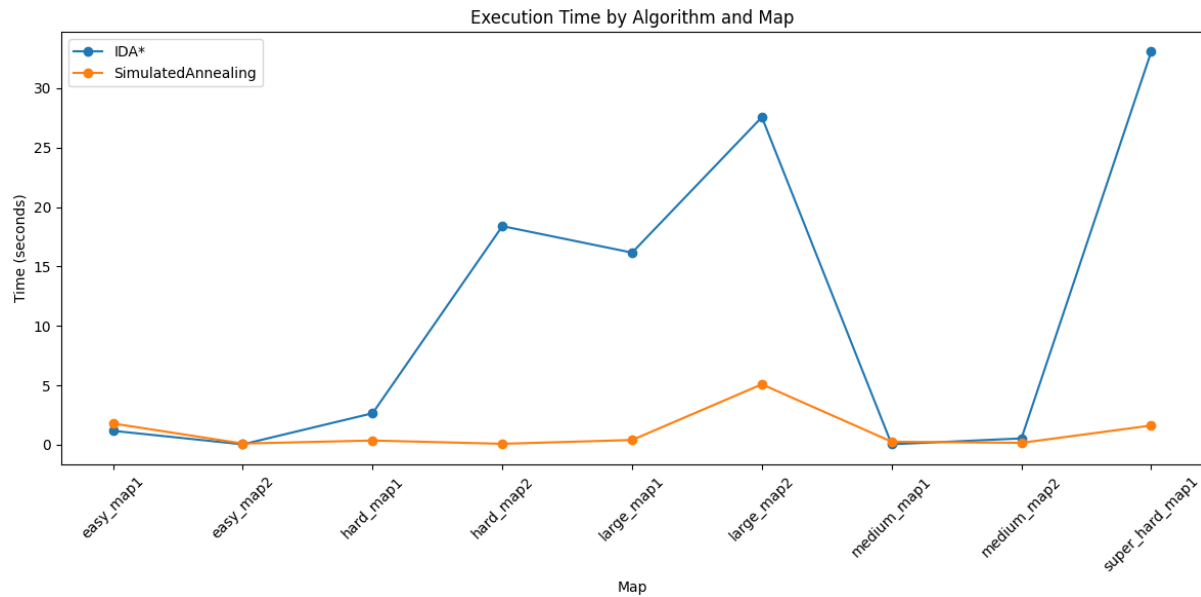
- `find_wall_deadlocks` care verifica daca exista cutii adiacente unui perete si pe acel perete nu exista niciun target (folosita doar atunci cand nu sunt admise miscari de pull)

Pentru a nu revizita starile am creat o functie `hash_state` care tine minte fiecare stare diferita in functie de pozitiile cutiilor si pozitia jucatorului. In simulated annealing am lasat posibila revizitarea starilor cu o probabilitate de 0.05 pentru evitarea blocarii in maxime locale.

Deoarece se permite uneori revizitarea starilor pot apara cicluri in solutie am creat functia `remove_cycles_from_solution` care verifica daca exista aceeasi stare de mai multe ori in solutie si elimina starile intermediare.

4. Comparatia algoritmilor si euristicilor

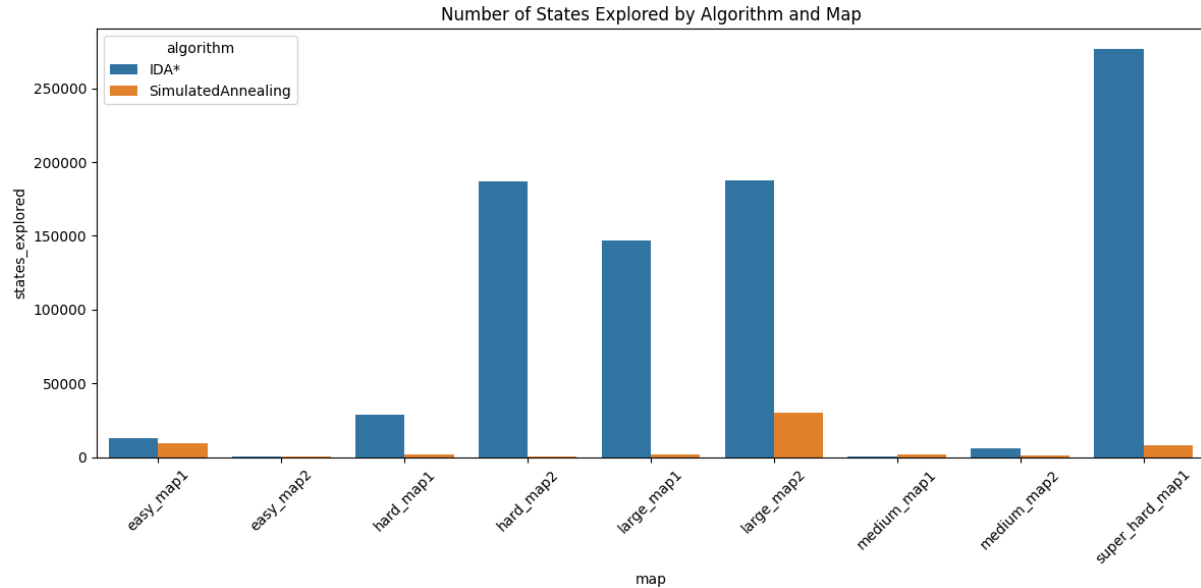
Timp de execuție



-IDA* are un timp de execuție mai lung mai ales pe hartile mai complexe

-Simulated Annealing oferă soluții mult mai rapid

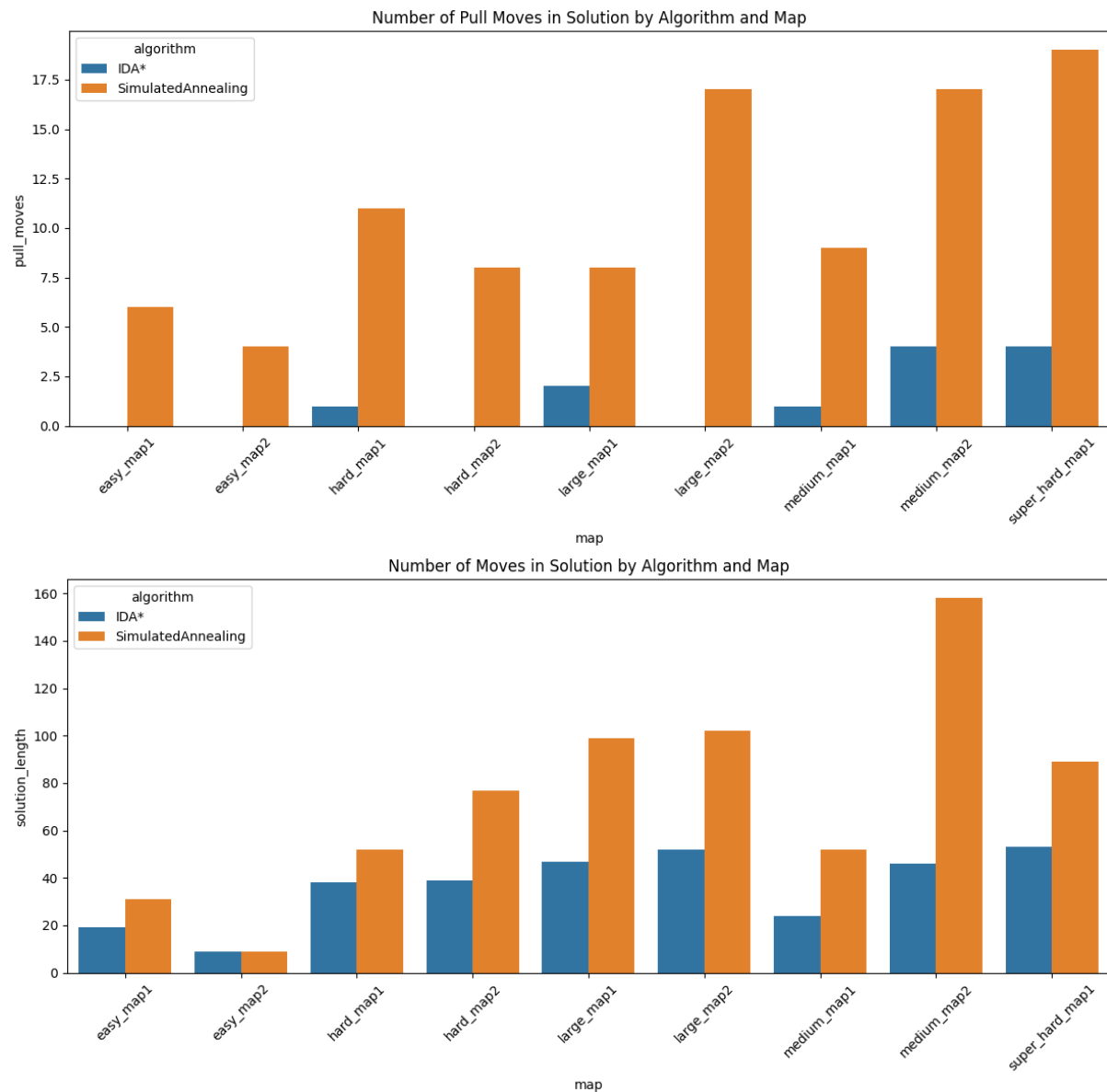
Număr de stări explorate



-IDA* explorează sistematic multe stări

-Simulated Annealing explorează aleator mai puține stări

Calitatea soluției

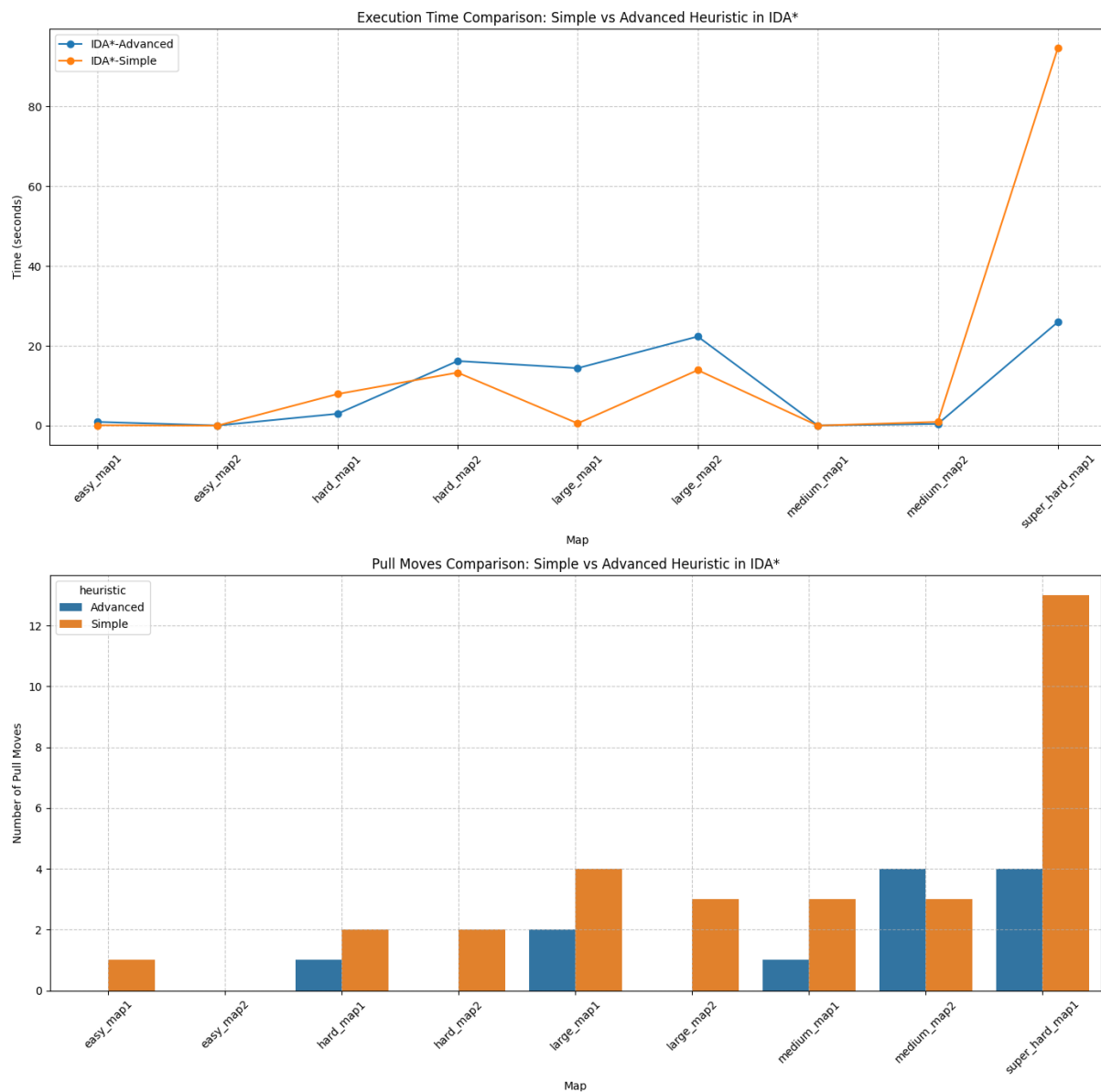


-IDA* găsește soluții mai scurte, cu mai puține mișcări de pull uneori chiar 0

-Simulated Annealing tinde să genereze soluții acceptabile, dar mai lungi și cu mult mai multe mișcări de pull

Comparatia performantei euristiciilor

Am ales sa compar diferenta intre euristici pentru algoritmul IDA* deoarece simulated annealing este influentat puternic de elemente aleatorii.



-Ambele euristici gasesc solutia intr-un timp similar, mai putin pentru ultima harta, fiind una foarte complexa, euristica avansata scade timpul de executie

-Euristica avansata gaseste solutii cu mult mai putine miscari de pull datorita penalizarii

5. Concluzii

Algoritmul IDA* este potrivit atunci cand avem nevoie de solutii cat mai bune deoarece incearca sa gaseasca solutia optima atunci cand foloseste o euristica admisibila. In plus, nu este influentat de elemente aleatorii garantand gasirea unei solutii.

Algoritmul simulated annealing este potrivit atunci cand avem nevoie de solutii cat mai rapide. Este util atunci cand spatial starilor este foarte mare deoarece exploreaza mai putine stari avand elemente aleatorii.